


```
package com.example.tests;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

/**
 * Sample TestNG Selenium test.
 * Demonstrates a simple login flow using stable locators.
 */
public class LoginTest {

    private WebDriver driver;
    private static final String BASE_URL = "https://example.com";

    @BeforeClass
    public void setUp() {
        // Set ChromeDriver path if not using WebDriverManager
        // System.setProperty("webdriver.chrome.driver", "/path/to/chromedriver");

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--headless"); // Run in headless mode; remove if UI is
        driver = new ChromeDriver(options);
    }
}
```

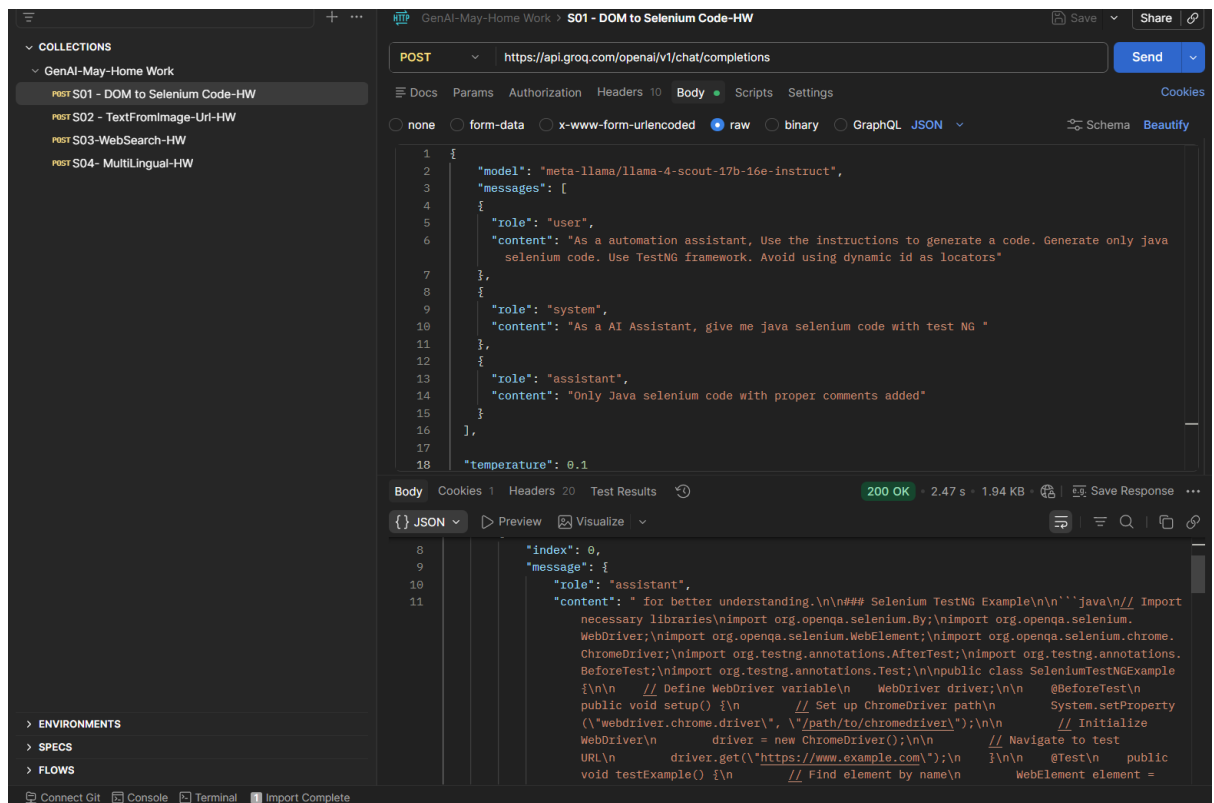
Message Copilot



Smart ▾



Using LLAMA



2. Text to image

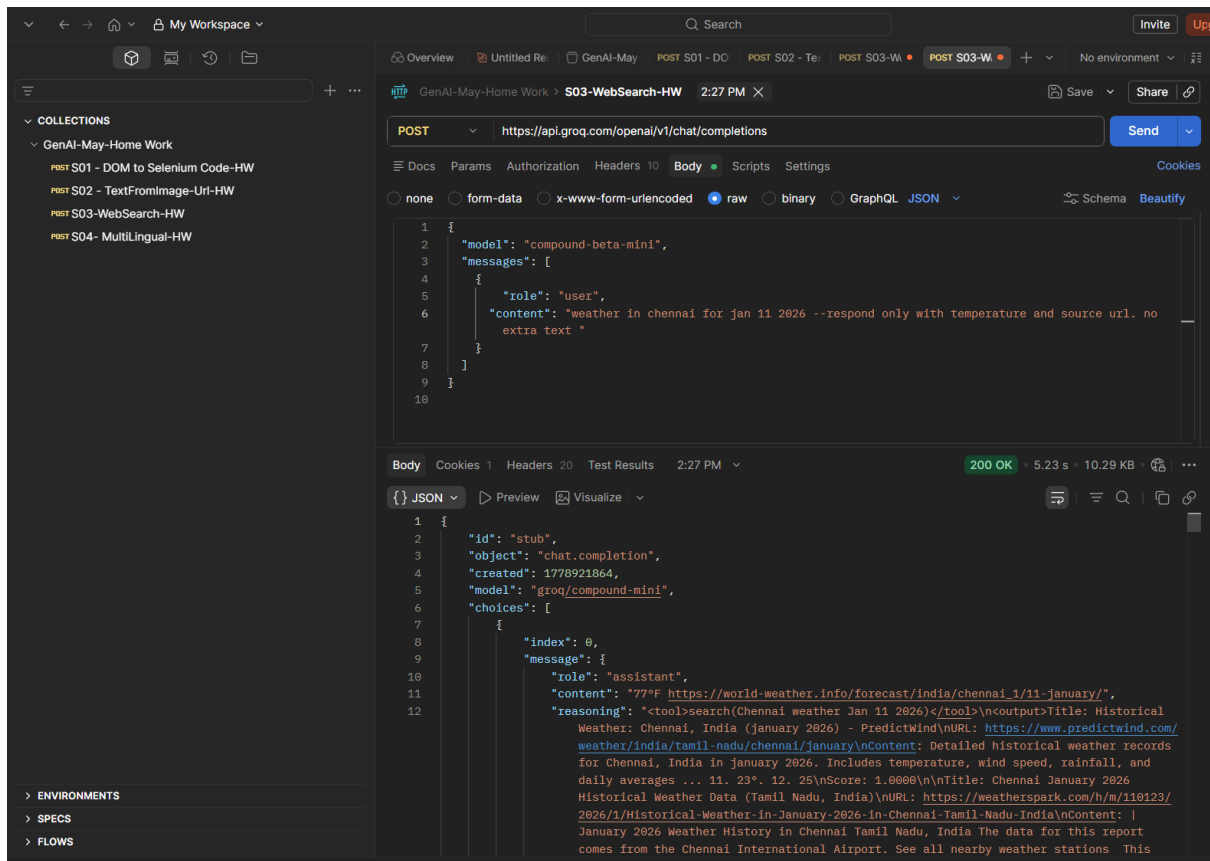
The screenshot displays a REST client interface with a workspace named "My Workspace". The left sidebar shows a collection of POST requests, with "POST S02 - TextFromImage-Url-HW" selected. The main panel shows the details of this request, which is a POST to `https://api.groq.com/openai/v1/chat/completions`. The request body is a JSON object with the following structure:

```
1 {
2   "model": "meta-llama/llama-4-scout-17b-16e-instruct",
3   "messages": [
4     {
5       "role": "user",
6       "content": [
7         {
8           "type": "text",
9           "text": "This is a image. Extract and return only the text exactly as shown, preserving case."
10        },
11        {
12          "type": "image_url",
13          "image_url": {
14            "url": "data:image/png;base64,
15              iVBORw0KGgoAAAANSUHEUGAAPIAAAVCAYAAADZyJdVAAAACXBIWMAAA7EAAAOxAGVKw4bAAAEc2lUWHRYTUw6Y2
16              9tLmFkb2JlLnhtcAAAAAAPD94cGFja2V0IGJlZ2l1PSFvu78nIGlkPSdXNU0wTXBDZWhpSHpyZVN6TlRjemtjOWQn
17              Pz4KPHg6eG1wbWV0YSB4bWxuczp4PSdhZG9iZTpuc3ptZXRhLyc
18              +CjxyZGY6UkRGIHhtbG5zOnJkZj0naHR0cDovL3d3dy53My5vcmcvMTk5OS8wMi8yMi1yZGYtc3ludG41LW5zIyc
19            }
20          }
21        ]
22      }
23    ]
24  }
```

The response is a JSON object with the following structure:

```
1 {
2   "id": "chatcmpl-073981f5-5ae4-4de0-bd4c-411d120551ad",
3   "object": "chat.completion",
4   "created": 1778922822,
5   "model": "meta-llama/llama-4-scout-17b-16e-instruct",
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "Testleaf \nAlways Ahead"
12      },
13      "logprobs": null,
14      "finish_reason": "stop"
15    }
16  ]
17 }
```

3. Web Search



4. Multilingual Search Tamil

My Workspace

COLLECTIONS

GenAI-May-Home Work

POST S01 - DOM to Selenium Code-HW

POST S02 - TextFromImage-Uri-HW

POST S03-WebSearch-HW

POST S04- MultiLingual-HW

ENVIRONMENTS

SPECS

FLOWES

GenAI-May-Home Work > S04- MultiLingual-HW

POSThttps://api.groq.com/openai/v1/chat/completions

DocsParamsAuthorizationHeaders10BodyScriptsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

```
1 {
2   "model": "llama-3.3-70b-versatile",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are an API Specialist. Always respond in the same language as the user."
7     },
8     {
9       "role": "user",
10      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்."
11    }
12  ]
13 }
```

BodyCookies1Headers20Test Results200 OK · 4.24 s · 1.63 KB

JSONPreviewVisualize

```
1 {
2   "id": "chatcmpl-ad32ff52-a043-4af9-9543-df088e642417",
3   "object": "chat.completion",
4   "created": 1778921997,
5   "model": "llama-3.3-70b-versatile",
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "வணக்கம்! நீங்கள் API-ஐ பயன்படுத்த விரும்புகிறீர்கள் என்பதை மிகவும் மகிழ்ச்சி
அடைகிறேன். இந்த API-ஐ பயன்படுத்துவது குறித்து விரிவாக விளக்குவேன்.\n\nமுதல் படி: A
குறியாக்கம் பெறுதல்**\nநீங்கள் API-ஐ பயன்படுத்த விரும்பினால், முதலில் ஒரு API
குறியாக்கத்தைப் பெற வேண்டும். இதற்கு, நீங்கள் எங்கள் இணையதளத்தில் பதிவு செய்ய வே
பதிவு செய்த பிறகு, நீங்கள் ஒரு API குறியாக்கத்தைப் பெறுவீர்கள்.\n\nஇரண்டாவது படி: AP
ஆவணங்களைப் படித்தல்**\nAPI குறியாக்கத்தைப் பெற்ற பிறகு, நீங்கள் API ஆவணங்களைப்
வேண்டும். இந்த ஆவணங்களில் API-ஐ பயன்படுத்துவதற்கான விரிவான வழிமுறைகள் இருக்கு
\n\nமூன்றாவது படி: API ஐ பயன்படுத்துதல்**\nAPI ஆவணங்களைப் படித்த பிறகு, நீங்கள் A
பயன்படுத்த தொடங்கலாம். நீங்கள் API குறியாக்கத்தைப் பயன்படுத்தி வழிமுறைகளை அனுப்ப
வழிமுறைகளை அனுப்பிய பிறகு, நீங்கள் ஒரு பதிலைப் பெறுவீர்கள்.\n\nநன்றி!+**\nநான் நீங்
```

Japanese

My Workspace

COLLECTIONS

GenAI-May-Home Work

POST S01 - DOM to Selenium Code-HW

POST S02 - TextFromImage-Uri-HW

POST S03-WebSearch-HW

POST S04- MultiLingual-HW

ENVIRONMENTS

SPECS

FLOWES

Search

GenAI-May-Home Work : S04- MultiLingual-HW

POST https://api.groq.com/openai/v1/chat/completions

Send

Docs Params Authorization Headers 10 Body Scripts Settings

Cookie

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "model": "llama-3.3-70b-versatile",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are an API Specialist. Always respond in the same language as the user."
7     },
8     {
9       "role": "user",
10      "content": "「APIの使い方」"
11    }
12  ]
13 }
```

Body Cookies 1 Headers 20 Test Results Save Response

JSON Preview Visualize

```
2 {"id": "chatcmpl-f99d0248-f5d5-413a-b370-d0ab8b42b444",
3  "object": "chat.completion",
4  "created": 1778922301,
5  "model": "llama-3.3-70b-versatile",
6  "choices": [
7    {
8      "index": 0,
9      "message": {
10       "role": "assistant",
11       "content": "APIの使い方は、以下の手順で一般的に行われます。
\n\n1. **APIの選択**: 使用したいAPIを選択します。さまざまなAPIが存在しますので、必要な機能を提供しているAPIを選択する必要があります。
\n2. **APIドキュメントの確認**: 選択したAPIのドキュメントを確認します。ドキュメントには、APIのエンドポイント、パラメータ、レスポンス形式などが記載されています。
\n3. **APIキーの取得**: APIを使うために、APIキーを取得する必要があります。APIキーは、APIプロバイダーが発行するUniqueなキーです。
\n4. **APIリクエストの送信**: APIキーを使用して、APIリクエストを送信します。リクエストには、エンドポイント、パラメータ、ヘッダーなどが含まれます。
\n5. **APIレスポンスの受信**: APIからレスポンスが返されます。レスポンスには、リクエストに応じたデータが含まれています。
\n6. **APIレスポンスの処理**: 受信したレスポンスを処理します。レスポンスは、JSON、XML、CSVなどさまざまな形式で返されます。
\n\n例えば、天気APIを使って、現在の天気情報を取得したい場合、以下の手順で行います。
\n\n* APIドキュメントを確認し、エンドポイントとパラメータを特定します。
\n* APIキーを取得します。
\n* エンドポイントとパラメータを使用して、APIリクエストを送信します。
```

Connect Git

Console

Terminal

Import Complete